Orchard Guide

The Zcash Orchard shielded-payment protocol

m@rek.onl

Abstract

This is the fourth and final volume of a four-part series — *Math Guide*, *Crypto Guide*, *Halo 2 Guide*, *Orchard Guide*. Assuming the three companion volumes, it develops Zcash's Orchard shielded-payment protocol: the key hierarchy, notes and note commitments, the commitment tree and the procedure by which a wallet obtains a membership path, nullifiers, note encryption and trial decryption, value commitments and the RedPallas signature, the formal Action statement the SNARK proves, and the end-to-end security analysis that derives balance and nullifier uniqueness from knowledge soundness, and privacy from zero knowledge — closing with the Zcash proof-system lineage.

Contents

1 Introduction

This is the *Orchard Guide*, the fourth and final volume of a development of the cryptography behind Zcash's Orchard protocol. It assumes the three companion volumes: the *Math Guide* (the underlying mathematics), the *Crypto Guide* (hash functions, commitments, signatures, key agreement, zero knowledge, and the remainder of the primitive toolbox), and the *Halo 2 Guide* (the general-purpose proof system). This volume assembles those tools into Zcash's deployed shielded-payment protocol.

We develop the protocol from the problems it solves. After we fix the curve-level preliminaries and the Sinsemilla hash, we develop each of the following as the answer to one sub-problem: how a unit of value is represented (notes and note commitments); how an owner proves ownership of a real, unspent note without revealing which one (the commitment tree and a membership path); how the protocol prevents double-spending (nullifiers); how a sender privately delivers and a recipient detects a note (note encryption and trial decryption); how value is conserved without disclosure (value commitments and the binding signature); and how an owner authorises a spend (RedPallas). We then show the procedure by which a node verifies a payment without ever seeing the note, state the Action relation formally, trace the end-to-end security reductions, and close the volume with the Zcash proof-system lineage from Sprout through Sapling to Orchard.

2 The Orchard application: the relation proved by the SNARK

Halo 2 is a general-purpose proving system; it has no notion of money. The Orchard application converts it into a shielded-payment protocol by fixing one specific relation — the *Action statement* — that every transaction must prove. This section develops that relation from the problem it solves rather than from the symbols it uses. It begins from the question of what a private payment must accomplish, derives each cryptographic object as the answer to one sub-problem, then traces a complete payment from Alice to Bob end to end, and only then states the Action relation formally.

2.1 The problem and the four requirements on a shielded payment

A transparent ledger (Bitcoin, Zcash's transparent pool) records, for every unspent output, its owner and its denomination (the number of units it holds), and a payment publicly transfers an output from one owner to another. A *shielded* payment must achieve the same effects — the transfer of value, the prevention of double-spending, and the conservation of total supply — while disclosing none of them. A private payment system must accordingly satisfy four requirements simultaneously:

1. *Represent a unit of value* so that its owner, value, and very existence stay hidden, yet its owner can later prove possession of it. (Notes and *note commitments* solve this, §2.5.)
2. *Prove possession of a real, unspent note* without identifying which note. (The *commitment tree* together with a membership proof solves this, §2.6.)
3. *Prevent double-spending* without a public list of which notes an owner spent. (*Nullifiers* solve this, §2.7.)
4. *Prove that no one created value* without revealing any amount. (*Value commitments* and the *binding signature* solve this, §2.9.)

These four requirements constitute the core of the protocol, and they rest on shared machinery. To render the section self-contained, we fix the build order by dependency — nothing appears before we define it — and lay it out here as a map:

1. **Preliminaries** (§2.2): the curve-level primitives every later object invokes — the hash-to-curve $\mathsf{GroupHash}$, the coordinate map $\mathsf{Extract}$, the point encoding $\star$, and domain separators.

2. **Sinsemilla** (§2.3): the commitment/hash built on those primitives, which both the note commitment and the Merkle tree use.

3. **Keys** (§2.4): the key hierarchy, from which the protocol derives addresses, commitments, and nullifiers.

4. **The first three requirements in turn**: notes (§2.5), the commitment tree (§2.6), and nullifiers (§2.7).

5. **Transmission** (§2.8): how the sender delivers the new note in-band and the recipient detects it.

6. **The fourth requirement**: value commitments (§2.9); together with *RedPallas* (§2.10), the signature that authorises a spend.

7. **Assembly** (§2.11): a complete Alice-to-Bob payment, end to end.

8. **The Action statement** (§2.13): the formal relation, the conjunction of in-circuit checks that makes all four hold at once.

Each later object depends only on earlier ones; the dependency root is the preliminaries below.

2.2 Preliminaries: hashing and encoding on the curve

The protocol builds every Orchard object below from a small set of curve-level primitives. We define them once, here, before anything uses them: a hash-to-curve $\mathsf{GroupHash}$, the coordinate map $\mathsf{Extract}$, the point encoding $\star$, and the domain separators that keep distinct uses independent. All operate on the Pallas group $\mathcal{E}(\mathbb{F}_{p_{\mathsf{Pallas}}})$.

The coordinate map $\mathsf{Extract}$. $\mathsf{Extract} : \mathcal{E}(\mathbb{F}_{p_{\mathsf{Pallas}}}) \to \mathbb{F}_{p_{\mathsf{Pallas}}}$ maps a curve point to its x-coordinate, $\mathsf{Extract}(x, y) := x$ (with $\mathsf{Extract}(\mathcal{O}) := 0$ by convention). It appears wherever a later object must be a *field element* rather than a curve point — because the Merkle tree, the nullifier set, and the proof's public inputs are all field elements. It is not injective ($\pm(x, y)$ share an x), which is harmless here: the protocol only ever requires the x-coordinate as a binding digest, never to recover the point.

Point encoding: the $\star$ superscript. $P^\star \in \{0,1\}^{256}$ denotes the canonical bit-encoding of a point $P \in \mathcal{E}(\mathbb{F}_{p_{\mathsf{Pallas}}})$: its x-coordinate (an element of $\mathbb{F}_{p_{\mathsf{Pallas}}}$, which fits in 255 bits, since $p_{\mathsf{Pallas}} < 2^{255}$) packed into a 256-bit string together with one sign bit, placed in the otherwise-unused top bit, recording which of the two square roots $\pm y$ is meant. Unlike $\mathsf{Extract}$, this encoding is injective — it determines P completely — and it is what we feed, as a bit-string, into a hash or commitment.

Integer encodings LE_n. $\mathrm{LE}_n(m) \in \{0,1\}^n$ denotes the n-bit little-endian encoding of an integer $m \in \{0, \ldots, 2^n - 1\}$: the bit string $b_0 b_1 \cdots b_{n-1}$ with $m = \sum_{i=0}^{n-1} b_i \, 2^i$ (a field element encodes via its integer representative). It renders integers and field elements as bit strings wherever they enter a hash or commitment: LE_{32} indexes the Sinsemilla generator table (§2.3), LE_{10} tags the Merkle-tree layers (§2.6), and LE_{64}, LE_{255} encode note values and field elements (§2.5).

The hash-to-curve $\mathsf{GroupHash}$. $\mathsf{GroupHash}^{\mathbb{G}} : \{0,1\}^* \times \{0,1\}^* \to \mathbb{G}$ takes a pair (domain separator D, message M) and returns a point of $\mathbb{G}$. It is *deterministic* (the same input always yields the same point), *public* (anyone can evaluate it), and *efficiently computable*. It instantiates the IETF hash-to-curve standard: hash_to_field expands (D, M) — via expand_message_xmd over BLAKE2b-512, with D embedded in the domain-separation tag — into *two* base-field elements;

each passes through the simplified SWU map onto a curve isogenous to $\mathbb{G}$; and the sum of the two resulting points is carried through the isogeny to $\mathbb{G}$ (the *Crypto Guide* develops this construction in full). For everything that follows, we use only the properties that $\mathsf{GroupHash}$ is deterministic, public, and behaves like a random choice of point.

That last property is the purpose of the construction: because every stage is public and deterministic, the public strings D, M pin down the output $\mathsf{GroupHash}^{\mathbb{G}}(D, M)$, so no party can have chosen it to satisfy a hidden relation such as $P_2 = [s]P_1$ for a secret s. We call such points *nothing-up-my-sleeve* generators. For the security arguments we *model* $\mathsf{GroupHash}$ as a random oracle into $\mathbb{G}$ (the *Crypto Guide*): we treat its outputs as independent uniform points, so the family it produces has no efficiently-findable non-trivial discrete-log relation $\sum_i [a_i]P_i = \mathcal{O}$ — precisely the assumption the Sinsemilla binding and collision-resistance reductions invoke (§3.2). As with all random-oracle arguments this is a strong, standard, but unprovable modelling heuristic about the concrete hash, not a theorem.

Domain separators. A *domain separator* $D \in \{0,1\}^*$ is a fixed ASCII byte string naming the use site, fed to $\mathsf{GroupHash}$ so that logically distinct uses draw independent, unrelated generators (a collision found in one context cannot be transported to another). The strings are part of the protocol specification; the ones relevant here are

`z.cash:Orchard`	fixed generators $G^{\mathsf{SpendAuth}}, G^{\mathsf{nf}}$ (§2.4, §2.7),
`z.cash:Orchard-gd`	the diversified base $\mathsf{g_d}$ (§2.4),
`z.cash:Orchard-NoteCommit`	note commitments (§2.5),
`z.cash:Orchard-CommitIvk`	the ivk commitment (§2.4),
`z.cash:Orchard-MerkleCRH`	the Merkle-tree node hash (§2.6),
`z.cash:Orchard-cv`	the value-commitment bases (§2.9),
`z.cash:SinsemillaQ`, `z.cash:SinsemillaS`	the Sinsemilla Q point and S table (§2.3).

2.3 The commitment primitive: Sinsemilla

Two of the objects we build — the note commitment (§2.5) and the Merkle-tree node hash (§2.6) — require the same primitive: a commitment/hash that the spender must recompute *inside the zero-knowledge circuit*. In-circuit cost is therefore the binding constraint, and it is what Sinsemilla (Bowe and Hopwood) is designed to minimise. Its collision resistance reduces to DL on a prime-order curve, so it introduces no assumption beyond the DL hardness that Orchard's algebraic core already requires. It builds directly on the preliminaries of §2.2: its generators are $\mathsf{GroupHash}$ outputs, and its short form returns an $\mathsf{Extract}$ed coordinate.

Construction 2.1 (Sinsemilla)**.** Fix a prime-order group $\mathbb{G}$ (Orchard: $\mathcal{E}(\mathbb{F}_{p_{\mathsf{Pallas}}})$) and a chunk width k ($k = 10$ in Orchard). From the domain separator D, derive the $2^k + 1$ Sinsemilla generators:

$$Q(D) := \mathsf{GroupHash}^{\mathbb{G}}(\texttt{z.cash:SinsemillaQ},\ D) \in \mathbb{G},$$
$$P[m] := \mathsf{GroupHash}^{\mathbb{G}}(\texttt{z.cash:SinsemillaS},\ \mathrm{LE}_{32}(m)) \in \mathbb{G}, \qquad 0 \le m < 2^k.$$

The point $Q(D)$ is the domain-dependent starting accumulator; the 2^k points $P[0], \ldots, P[2^k - 1]$ form a fixed table indexed by a k-bit chunk value, shared across all Sinsemilla domains. For a message M split into k-bit chunks $m_1, \ldots, m_\ell$ (so each $m_i \in \{0, \ldots, 2^k - 1\}$ indexes the table),

$$\mathsf{Sinsemilla}_D(M) := \mathsf{Acc}_\ell, \quad \mathsf{Acc}_0 := Q(D), \quad \mathsf{Acc}_{i+1} := (\mathsf{Acc}_i \mathbin{\dot{+}} P[m_{i+1}]) \mathbin{\dot{+}} \mathsf{Acc}_i,$$

with $\dot{+}$ *incomplete* short-Weierstrass addition. The output $\mathsf{Acc}_\ell \in \mathbb{G}$ is a curve point.

Theorem 2.2 (Sinsemilla collision resistance)**.** *For fixed-length inputs,* $\mathsf{Sinsemilla}_D$ *is collision-resistant assuming DL on* $\mathbb{G}$ *and modelling the generator derivation as a random oracle. (We prove this in §3.2.)*

Definition 2.3 (Sinsemilla commitment and short commitment)**.** The *Sinsemilla commitment* adds a hiding term to the hash:

$$\mathsf{SinsemillaCommit}_r(D, M) := \mathsf{Sinsemilla}_{D\,\|\,\texttt{-M}}(M) \dot{+} [r]\, H_D \;\in\; \mathbb{G},$$

where $r \in \mathbb{F}_{p_{\mathsf{Vesta}}}$ is the commitment randomness (trapdoor), the hash runs in the suffixed domain $D \,\|\, \texttt{-M}$, and $H_D \in \mathbb{G}$ is a fixed blinding generator for the domain D, derived as the GroupHash output

$$H_D := \mathsf{GroupHash}^{\mathbb{G}}\big(D \,\|\, \texttt{-r},\ \varepsilon\big) \in \mathbb{G}$$

— the suffix `-r` appended to the domain string, with the empty message ε (a nothing-up-my-sleeve point, §2.2). Because H_D is a GroupHash output in its own sub-domain, no party knows a discrete-log relation between it and the generators $Q(D \,\|\, \texttt{-M}), P[m]$ that the hash uses; this independence is exactly what keeps the commitment binding — a party knowing such a relation could trade the blinding term against the message and open one commitment to two distinct messages — while hiding needs no assumption at all: for uniform r the term $[r]H_D$ is uniform on $\mathbb{G}$ and masks the hash completely. This is the same division of roles the independent base H supports in a Pedersen commitment (the *Crypto Guide*). The *short commitment* returns only its x-coordinate, a base-field element rather than a curve point:

$$\mathsf{ShortCommit}_r(D, M) := \mathsf{Extract}\big(\mathsf{SinsemillaCommit}_r(D, M)\big) \;\in\; \mathbb{F}_{p_{\mathsf{Pallas}}},$$

with Extract the coordinate map of §2.2. The unkeyed *short hash* omits the blinding term altogether:

$$\mathsf{ShortHash}_D(M) := \mathsf{Extract}\big(\mathsf{Sinsemilla}_D(M)\big) \;\in\; \mathbb{F}_{p_{\mathsf{Pallas}}},$$

the unblinded analogue of ShortCommit — no randomness argument, collision-resistant (Theorem 2.2) but not hiding.

The commitment is perfectly hiding and computationally binding — the two properties §2.5 requires of NoteCommit. Indeed the note commitment $\mathsf{NoteCommit}_{\mathsf{rcm}}(\cdot)$ is exactly $\mathsf{SinsemillaCommit}_{\mathsf{rcm}}$ under the domain $\texttt{z.cash} : \texttt{Orchard-NoteCommit}$. The short forms appear wherever the consumer requires a field element instead of a point: ShortHash in the Merkle-tree node hash (§2.6), ShortCommit in the incoming viewing key ivk (§2.4). The latter use enters through a named instance that fixes the domain and the message encoding:

$$\mathsf{Commit}^{\mathsf{ivk}}_{\mathsf{r_{ivk}}}(x, y) := \mathsf{ShortCommit}_{\mathsf{r_{ivk}}}\big(\texttt{z.cash} : \texttt{Orchard-CommitIvk},\ \mathrm{LE}_{255}(x) \,\|\, \mathrm{LE}_{255}(y)\big),$$

taking two field elements and concatenating their 255-bit little-endian encodings into the 510-bit message. Its output lies in $\{1, \ldots, p_{\mathsf{Pallas}} - 1\}$ (the zero output cannot occur for a valid key, so ivk is a usable scalar).

In-circuit cost. Each round consumes a whole k-bit chunk via one table lookup of $P[m]$ and two incomplete additions, so a 510-bit message costs 51 rounds rather than 510 bit-operations. In a PLONKish circuit with lookups (the *Halo 2 Guide*) this is dramatically cheaper than Sapling's bit-by-bit Pedersen hash — which is the reason the protocol changed hash between generations.

2.4 Keys: capabilities for spending and for viewing

We build the Orchard machinery bottom-up, beginning with the keys, because everything later — addresses, note commitments, nullifiers — derives from them. A practical requirement shapes the key hierarchy: different parties should be able to perform different operations. The owner can spend. An auditor should be able to *see* incoming and outgoing payments without being able to spend. A point-of-sale terminal should be able to *detect* incoming payments without being able to see outgoing ones. Each of these is a distinct capability, and the key tree makes each capability a separate derived key — the owner can hand out a lower-level key without surrendering the powers above it.

Definition 2.4 (Orchard key hierarchy)**.** The root secret is a 32-byte *spending key* sk, chosen uniformly at random when the owner creates the account (in a deployed wallet it typically derives from a seed phrase via ZIP-32 hierarchical-deterministic derivation, but for present purposes it is simply a uniform 256-bit string, $\mathsf{sk} \in \{0,1\}^{256}$). Every other key is a deterministic function of sk.

Expansion function. The expansion uses BLAKE2b-512, a conventional (non-arithmetisation-friendly) cryptographic hash with a 512-bit (64-byte) digest; we write PRF-style expressions with two arguments, the first a key and the second a message. A *pseudorandom function* (PRF) is a keyed family $\{f_K\}_K$ whose outputs are, for a secret uniform K, computationally indistinguishable from those of a truly random function; here the protocol obtains one not from BLAKE2b's native keyed mode but as *personalised, unkeyed* BLAKE2b over the concatenation $\mathsf{sk} \,\|\, t$, a PRF of t whenever sk is secret.

Accordingly, write $\mathsf{PRFexpand} : \{0,1\}^{256} \times \{0,1\}^* \to \{0,1\}^{512}$ for the PRF instantiated as $\mathsf{PRFexpand}(\mathsf{sk}, t) :=$ BLAKE2b-512(``Zcash_ExpandSeed'', $\mathsf{sk} \,\|\, t$) — unkeyed BLAKE2b-512 with the fixed 16-byte personalisation string ``Zcash_ExpandSeed'' and input $\mathsf{sk} \,\|\, t$, where the secret sk acts as the PRF key and the leading bytes of t are a domain-separation tag identifying which sub-secret it produces. (We write the key as a subscript, $\mathsf{PRFexpand}_{\mathsf{sk}}(t)$, when convenient.) We reduce its 512-bit output into the required field or scalar set. From sk derive the three spend-side secrets

$$\begin{aligned} \mathsf{ask} &:= \mathrm{ToScalar}\big(\mathsf{PRFexpand}_{\mathsf{sk}}(\mathtt{0x06})\big) \in \mathbb{F}_{p_{\mathsf{Vesta}}}, \\ \mathsf{nk} &:= \mathrm{ToBase}\big(\mathsf{PRFexpand}_{\mathsf{sk}}(\mathtt{0x07})\big) \in \mathbb{F}_{p_{\mathsf{Pallas}}}, \\ \mathsf{r_{ivk}} &:= \mathrm{ToScalar}\big(\mathsf{PRFexpand}_{\mathsf{sk}}(\mathtt{0x08})\big) \in \mathbb{F}_{p_{\mathsf{Vesta}}}, \end{aligned}$$

so that the *spend authorising key* ask and the *CommitIvk randomness* $\mathsf{r_{ivk}}$ are scalars in $\mathbb{F}_{p_{\mathsf{Vesta}}}$ (the scalar field of Pallas, i.e. the integers mod the Pallas group order p_{Vesta}), while the *nullifier key* nk is a base-field element in $\mathbb{F}_{p_{\mathsf{Pallas}}}$. The two reduction maps read the 512-bit PRF output as an integer and reduce it modulo the relevant prime:

$$\begin{aligned} \mathrm{ToScalar}(x) &:= \mathrm{LEOS2IP}_{512}(x) \bmod p_{\mathsf{Vesta}} \;\in\; \mathbb{F}_{p_{\mathsf{Vesta}}}, \\ \mathrm{ToBase}(x) &:= \mathrm{LEOS2IP}_{512}(x) \bmod p_{\mathsf{Pallas}} \;\in\; \mathbb{F}_{p_{\mathsf{Pallas}}}, \end{aligned}$$

where $\mathrm{LEOS2IP}_{512}$ is the little-endian octet-string-to-integer map sending a 64-byte string to the integer $\sum_{i=0}^{63} x_i\, 256^i \in \{0, \ldots, 2^{512}-1\}$. ToScalar lands in $\mathbb{F}_{p_{\mathsf{Vesta}}}$, ToBase in $\mathbb{F}_{p_{\mathsf{Pallas}}}$; the two targets differ only because the protocol uses nk as a field element (it feeds it to the nullifier PRF, §2.7) whereas it uses $\mathsf{ask}, \mathsf{r_{ivk}}$ as scalars multiplying group elements. Because the input is 512 bits wide while each modulus is just over 2^{254} (of bit length 255), the reduction is statistically indistinguishable from uniform on the target set (the modular bias is $O(2^{254}/2^{512}) = O(2^{-258})$, negligible). The distinct tag bytes $\mathtt{0x06}, \mathtt{0x07}, \mathtt{0x08}$ make the three outputs independent.

Spend validating key. Let $G^{\mathsf{SpendAuth}} \in \mathcal{E}(\mathbb{F}_{p_{\mathsf{Pallas}}})$ be a fixed, publicly-known generator of the Pallas group, obtained by $G^{\mathsf{SpendAuth}} := \mathsf{GroupHash}^{\mathsf{Pallas}}(\mathtt{z.cash:Orchard}, \mathtt{G})$ (a nothing-up-my-sleeve point, §2.2). The *spend validating key* is then the public point

$$\mathsf{ak} := [\mathsf{ask}]\, G^{\mathsf{SpendAuth}} \in \mathcal{E}(\mathbb{F}_{p_{\mathsf{Pallas}}}).$$

Key generation canonicalises its sign: if $[\mathsf{ask}]\, G^{\mathsf{SpendAuth}}$ has odd y-coordinate, ask is replaced by $-\mathsf{ask}$, so that ak always has even y and the x-coordinate $\mathsf{Extract}(\mathsf{ak})$ alone determines the point. The *full viewing key* bundles the three quantities needed to recognise and decrypt one's own notes, $\mathsf{fvk} := (\mathsf{ak}, \mathsf{nk}, \mathsf{r_{ivk}}) \in \mathcal{E}(\mathbb{F}_{p_{\mathsf{Pallas}}}) \times \mathbb{F}_{p_{\mathsf{Pallas}}} \times \mathbb{F}_{p_{\mathsf{Vesta}}}$. From it derive

$$\begin{aligned} \mathsf{ivk} &:= \mathsf{Commit}^{\mathsf{ivk}}_{\mathsf{r_{ivk}}}\big(\mathsf{Extract}(\mathsf{ak}),\ \mathsf{nk}\big) \in \{1, \ldots, p_{\mathsf{Pallas}} - 1\} \subset \mathbb{F}_{p_{\mathsf{Pallas}}}, \\ K &:= \mathrm{LE}_{256}(\mathsf{r_{ivk}}) \in \{0,1\}^{256}, \\ R &:= \mathsf{PRFexpand}\big(K,\ \mathtt{0x82} \,\|\, \mathrm{I2LEOSP}_{256}(\mathsf{ak}) \,\|\, \mathrm{I2LEOSP}_{256}(\mathsf{nk})\big) \in \{0,1\}^{512}, \\ (\mathsf{dk}, \mathsf{ovk}) &:= \big(R_{[0\,..\,256)},\ R_{[256\,..\,512)}\big) \in \{0,1\}^{256} \times \{0,1\}^{256}, \end{aligned}$$

where $\mathsf{Commit}^{\mathsf{ivk}}$ is the named ShortCommit instance of §2.3 and LE_{256} the 256-bit little-endian bit encoding of §2.2, giving K the type $\{0,1\}^{256}$ that PRFexpand requires of its key. This yields the *incoming* viewing key ivk (a base-field scalar), the *diversifier key* dk, and the *outgoing* viewing key ovk. The dk/ovk derivation is precisely the following: we split a *single* 512-bit PRFexpand output R, keyed by $\mathsf{r}_{\mathsf{ivk}}$ over the input $\mathtt{0x82} \,\|\, \mathsf{ak} \,\|\, \mathsf{nk}$ (not over an empty message), into its two 256-bit halves — dk the first, ovk the second. The diversifier d below is *not* part of this split; it derives afterwards from dk. Here $\mathsf{Extract}(\mathsf{ak})$ is the x-coordinate of ak (§2.2) — which, after the sign canonicalisation above, determines ak — while $\mathrm{I2LEOSP}_{256}$ denotes the 32-byte little-endian encoding of a field element or point. A *diversified address* is, for any index $d_{\mathrm{plain}} \in \{0,1\}^{88}$,

$$
\begin{aligned}
d &:= \mathsf{FF1\text{-}AES256}_{\mathsf{dk}}(d_{\mathrm{plain}}) \in \{0,1\}^{88},\\
\mathsf{g_d} &:= \mathsf{GroupHash}^{\mathsf{Pallas}}(\mathtt{z.cash} : \mathtt{Orchard\text{-}gd}, d) \in \mathcal{E}(\mathbb{F}_{p_{\mathsf{Pallas}}}),\\
\mathsf{pk_d} &:= [\mathsf{ivk}]\,\mathsf{g_d} \in \mathcal{E}(\mathbb{F}_{p_{\mathsf{Pallas}}}),
\end{aligned}
$$

where $\mathsf{FF1\text{-}AES256}_{\mathsf{dk}}$ produces the *diversifier* $d \in \{0,1\}^{88}$, the NIST FF1 format-preserving encryption mode (built on AES-256) keyed by dk: a keyed *permutation* of the 88-bit index space $\{0,1\}^{88}$, so that each d_{plain} maps to a distinct 88-bit diversifier d and different d_{plain} give unlinkable-looking addresses, all derived from the one key. The *address* is the pair $\mathsf{addr} := (d, \mathsf{pk_d}) \in \{0,1\}^{88} \times \mathcal{E}(\mathbb{F}_{p_{\mathsf{Pallas}}})$.

Choosing the index d_{plain}. The receiving wallet chooses the index freely: *any* of the 2^{88} values yields a valid, spendable address for the same ivk. No derivation fixes it and no index is forbidden — any 88-bit string is a valid diversifier index. The wallet allocates indices on demand, conventionally sequentially from $d_{\mathrm{plain}} = 0$ (so index 0 is the *default diversifier*), assigning a fresh index per payee or per invoice to keep their addresses mutually unlinkable; a randomly drawn index would yield an equally valid address, but the specification states that diversifier indices SHOULD NOT be chosen at random: ZIP-32 specifies their usage in hierarchical-deterministic wallets, whose seed-only recovery of issued addresses depends on deterministic allocation. Every index works for a reason specific to Orchard: the hash-to-curve $\mathsf{GroupHash}^{\mathsf{Pallas}}$ of §2.2 is total — it returns a point for every input — so every index yields a well-defined $\mathsf{g_d}$; in the negligible-probability event that the two summed SWU outputs cancel to $\mathcal{O}$, the specification substitutes the fallback $\mathsf{GroupHash}(D, \mathtt{""})$. This is a deliberate improvement over Sapling, where the diversifier hash failed on roughly half of all indices and the wallet had to increment d_{plain} until it found a valid one; in Orchard no such rejection loop exists.

Rationale for routing the index through AES. The step from d_{plain} to d is a keyed *permutation* (format-preserving encryption) for three reasons that no simpler map satisfies together. (i) *Unlinkability:* wallets allocate indices in order $0, 1, 2, \ldots$, and consecutive diversifiers would reveal a shared owner and an issue count; encryption under dk scatters them pseudorandomly over $\{0,1\}^{88}$. (ii) *Bijection:* a permutation never collides two indices onto one d and is invertible — with dk the wallet recovers the index behind any of its addresses, storing no per-address secret; a plain hash is neither collision-free nor invertible. (iii) *Format preservation:* the address format fixes the diversifier at exactly 88 bits, and FF1 maps $\{0,1\}^{88} \to \{0,1\}^{88}$ bijectively, which a truncated hash cannot. This derivation runs *wallet-side, never in the circuit*, so a conventional fast cipher (AES-256 under NIST FF1) is appropriate; arithmetisation-friendliness is irrelevant here.

The structure is a deliberate capability ladder (Figure 1), each rung conferring strictly less power than the one above:

- sk / ask — *spend.* Whoever holds ask can authorise spends; it sits at the top and nothing below it can recover it.
- fvk (hence ivk, ovk) — *see everything.* Detects incoming notes (via ivk, which recovers $\mathsf{pk_d}$) and views outgoing notes (via ovk), but cannot spend, because deriving fvk from sk is one-way.

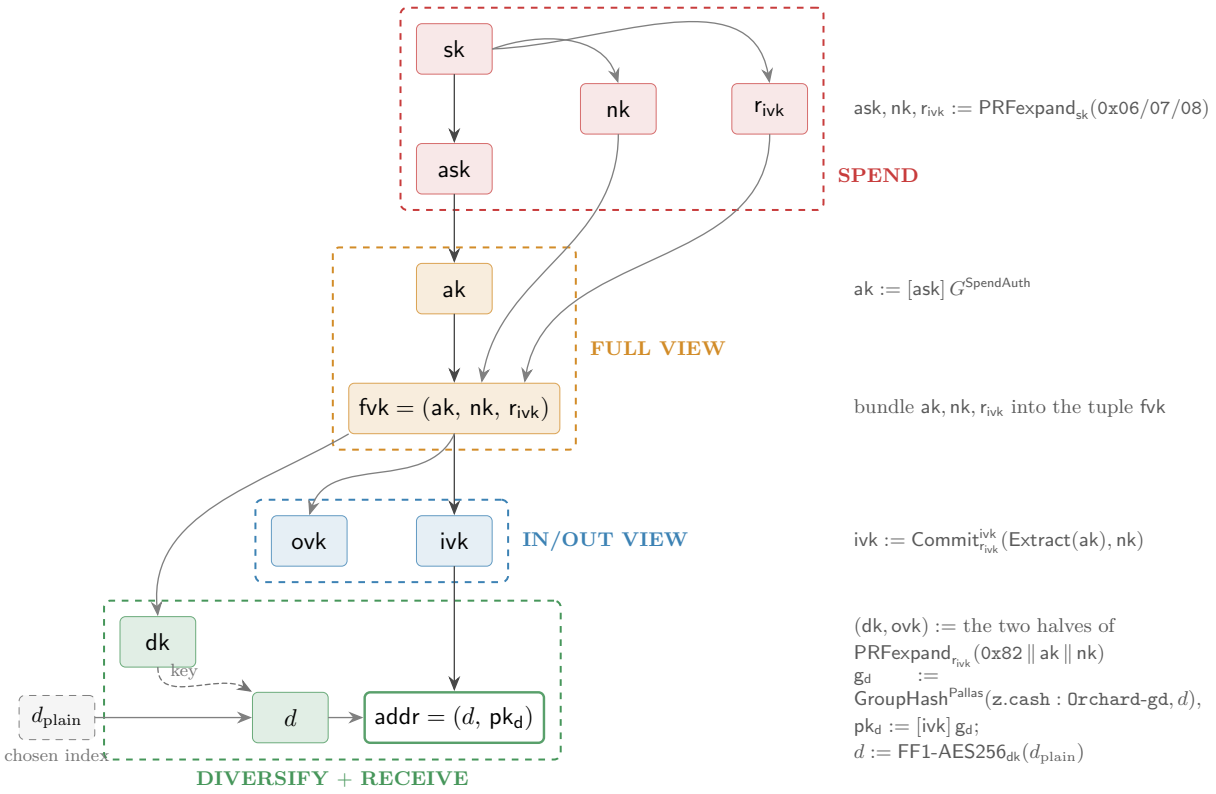

Figure 1: The Orchard key hierarchy as a capability ladder, read top to bottom. Each arrow is a derivation; the right-hand gutter shows the operation that produces the child from its parent(s). Not every edge is one-way: the bundling edges ($\mathsf{ak}, \mathsf{nk}, \mathsf{r_{ivk}}$ into the tuple fvk; d into the address) invert by projection, and the FF1 edge is a keyed permutation the dk holder inverts — the tiers are separated by the one-way steps $\mathsf{sk} \to (\mathsf{ask}, \mathsf{nk}, \mathsf{r_{ivk}})$, $\mathsf{ask} \to \mathsf{ak}$, $\mathsf{fvk} \to (\mathsf{ivk}, \mathsf{dk}, \mathsf{ovk})$, and $\mathsf{ivk} \to \mathsf{pk_d}$. The dk and ovk keys are the two halves of one $\mathsf{PRFexpand}_{\mathsf{r_{ivk}}}$ output (tag $\mathsf{0x82}$, input $\mathsf{ak}\|\mathsf{nk}$); they play different roles and land in different tiers — ovk is the *outgoing* viewing key, grouped with the *incoming* viewing key ivk as the IN/OUT viewing tier, whereas dk is the *diversifier* key, which sits in the address layer because it generates diversified addresses. Concretely d is the format-preserving encryption $\mathsf{FF1\text{-}AES256}$ of a chosen index d_{plain} under the key dk (the dashed edge marks dk entering as the cipher key), and $\mathsf{pk_d} = [\mathsf{ivk}]\,\mathsf{g_d}$, so the address binds both keys. *Every node is secret key material except the chosen index d_{plain}, the diversifier d, and the diversified address (unshaded) — the address $(d, \mathsf{pk_d})$, and with it d, is the only value the protocol ever publishes* (shading marks tier membership, not secrecy) — in particular the viewing keys fvk, ivk, ovk are *secret:* "viewing" names what they let the holder do, not that the protocol publishes them. Each dashed tier holds a strictly weaker capability than the one above, so a holder may receive a lower-tier key without gaining the powers above it. Drawn to match Definition 2.4.

- ivk alone — *detect incoming.* Sufficient to recognise notes addressed to the holder and scan the chain, nothing more — the key suitable for an always-online wallet server.
- A diversified address $(d, \mathsf{pk_d})$ — *receive.* A public handle anyone can pay; one ivk generates 2^{88} mutually unlinkable addresses — one per diversifier index (Definition 2.4), practically inexhaustible — all spendable by the same key.

Of these, only the diversified address is *public.* The viewing keys $\mathsf{fvk}, \mathsf{ivk}, \mathsf{ovk}$ are *secret* key material — "viewing" names what they let the holder *do* (see transactions), not that one may publish them. Disclosing a viewing key confers the corresponding visibility on the recipient — every payment to and from the wallet for fvk, incoming payments only for ivk, outgoing only for ovk — so the owner shares one only deliberately (with an auditor, say) and otherwise guards it like any other secret. The ladder is useful precisely because these capabilities are *separable*: one can delegate "see incoming" (ivk) without delegating "see everything" (fvk), and neither confers the ability to spend.

Two questions of motivation that the bare formulas do not answer. *Why is* ivk *a commitment,* $\mathsf{Commit}^{\mathsf{ivk}}_{\mathsf{r_{ivk}}}(\mathsf{Extract}(\mathsf{ak}), \mathsf{nk})$*, rather than a plain hash?* Because ivk must both bind *and* hide. It binds ak (spend authority) and nk (nullifier derivation) to one identity, and check A7 below proves the linkage in-circuit by recomputing the commitment from the witnessed ak, nk, $\mathsf{r_{ivk}}$ — but binding alone a collision-resistant hash would supply equally, and in-circuit recomputation works for any hash (check A3 recomputes the unblinded Sinsemilla $\mathsf{MerkleCRH}$; Sapling even proved its BLAKE2s-based $\mathsf{CRH}^{\mathsf{ivk}}$ in-circuit). What the trapdoor $\mathsf{r_{ivk}}$ adds is *hiding*: the blinded $\mathsf{ShortCommit}$ is perfectly hiding, so ivk — and hence every public $\mathsf{pk_d} = [\mathsf{ivk}]\mathsf{g_d}$ — reveals nothing about ak and nk, whereas the unblinded Sinsemilla hash is only collision-resistant (Theorem 2.2) with no one-wayness claim. Hiding is what keeps the IN/OUT tier strictly below FULL VIEW: an ivk holder, such as an always-online wallet server, cannot extract nk or ak. *Why is* nk *separate from* ask*?* So that the owner can delegate the capability to compute nullifiers (which a viewing key needs to determine which of the holder's notes are already spent) *without* conferring the ability to spend — the viewing key holds nk but never ask.

2.5 Representation of a note: notes and note commitments

The first requirement is to represent a note such that it stays invisible on-chain yet provable by its owner. The protocol therefore never places the note on the chain at all; it publishes only a hiding commitment to it.

Definition 2.5 (Orchard note)**.** An Orchard *note* — the private object representing one unit of value — is a tuple

$$\mathbf{n} := (\mathsf{addr},\ v,\ \rho,\ \psi,\ \mathsf{rcm}),$$

where $\mathsf{addr} := (d, \mathsf{pk_d})$ is the recipient's diversified address; $v \in \{0, \ldots, 2^{64} - 1\}$ is the value in zatoshi; $\rho \in \mathbb{F}_{p_{\mathsf{Pallas}}}$ is a per-note uniqueness seed; $\psi \in \mathbb{F}_{p_{\mathsf{Pallas}}}$ is randomness the sender fixes through the note seed rseed, from which it is derived deterministically (§2.8); and $\mathsf{rcm} \in \mathbb{F}_{p_{\mathsf{Vesta}}}$ is the commitment trapdoor.

The protocol never publishes the note itself. What it records on-chain is its *note commitment*

$$\mathsf{cm} := \mathsf{NoteCommit}_{\mathsf{rcm}}\big(\mathsf{g_d}^{\star} \,\|\, \mathsf{pk_d}^{\star} \,\|\, \mathrm{LE}_{64}(v) \,\|\, \mathrm{LE}_{255}(\rho) \,\|\, \mathrm{LE}_{255}(\psi)\big) \ \in\ \mathcal{E}(\mathbb{F}_{p_{\mathsf{Pallas}}}),$$

a commitment that is *perfectly hiding* (the chain learns nothing about $\mathsf{addr}, v, \rho, \psi$ from cm) and *computationally binding* (the sender cannot later claim the commitment was to a different note). These are precisely the two properties of a commitment scheme from the *Crypto Guide*; here the scheme is Sinsemilla (§2.3), chosen because it is inexpensive to recompute *inside the circuit*, which the spender must do. The chain stores $\mathsf{cmx} := \mathsf{Extract}(\mathsf{cm}) \in \mathbb{F}_{p_{\mathsf{Pallas}}}$, the x-coordinate of the commitment point, because the Merkle tree consumes a field element.

The three address-related quantities appearing in NoteCommit are the key material that §2.4 defines: the diversifier $d \in \{0,1\}^{88}$, the diversified base $\mathsf{g_d} \in \mathcal{E}(\mathbb{F}_{p_{\mathsf{Pallas}}})$ (the per-address generator $\mathsf{GroupHash}^{\mathsf{Pallas}}(\texttt{z.cash} : \texttt{Orchard-gd}, d)$), and the transmission key $\mathsf{pk_d} = [\mathsf{ivk}]\,\mathsf{g_d} \in \mathcal{E}(\mathbb{F}_{p_{\mathsf{Pallas}}})$ that identifies the recipient; the address is $\mathsf{addr} = (d, \mathsf{pk_d})$. Recall the $\star$-encoding from §2.2: $\mathsf{g_d}^\star, \mathsf{pk_d}^\star \in \{0,1\}^{256}$. The NoteCommit input is the bit-string concatenation ($\|$) of these point encodings with the little-endian integer encodings $\mathrm{LE}_{64}(v) \in \{0,1\}^{64}$ and $\mathrm{LE}_{255}(\rho), \mathrm{LE}_{255}(\psi) \in \{0,1\}^{255}$.

The choice of fields is as follows. addr and v are the note's content. ρ and ψ exist solely to render the eventual nullifier unique and unpredictable; their role becomes apparent only in §2.7: a note carries randomness whose purpose is downstream.

2.6 Proof of ownership of *some* valid note: the commitment tree

The second requirement is to prove the spent note is valid without revealing which note. When Alice spends, she must convince the network that the note she is spending is a valid note that someone actually created; otherwise she could spend a note she invented. On a transparent chain this is trivial: the output is present in the UTXO set. But Alice must prove it *without revealing which note*, because revealing the note would deanonymise the payment.

The network maintains one append-only Merkle tree whose leaves are *every note commitment ever published*, in creation order. Alice proves *membership*: a Merkle authentication path from her leaf cmx to a root the network trusts. Inside the circuit the path is a private witness (check A3), so the verifier learns that her note is one of the leaves but not which one; the anonymity set is the entire tree.

Definition 2.6 (Orchard commitment tree)**.** The Orchard *note commitment tree* is an append-only Merkle tree of fixed depth 32 (so up to 2^{32} notes). Leaves are cmx values in creation order. The parent of a left child ℓ and right child r at layer ℓ' is

$$\mathsf{MerkleCRH}^{\mathsf{Orchard}}_{\ell'}(\ell, r) := \mathsf{ShortHash}_{\texttt{z.cash:Orchard-MerkleCRH}}(\mathrm{LE}_{10}(\ell') \,\|\, \mathrm{LE}_{255}(\ell) \,\|\, \mathrm{LE}_{255}(r)) \in \mathbb{F}_{p_{\mathsf{Pallas}}},$$

a layer-tagged Sinsemilla short hash (Definition 2.3) of the $10 + 255 + 255 = 520$-bit message — 52 ten-bit chunks, the same LE-encoded input convention as $\mathsf{Commit}^{\mathsf{ivk}}$ and NoteCommit (leaves cmx, internal nodes, and the root all live in $\mathbb{F}_{p_{\mathsf{Pallas}}}$). Appending a block's note commitments yields a new root: the root of the tree after block h is the *anchor* of block h (a single field element), so every main-chain block defines exactly one anchor. A spend proves membership against the anchor of any main-chain block, and all Actions of one bundle prove against a single shared anchor (§2.12).

Every node is a single base-field element by necessity, not convention. The spender recomputes this hash layer by layer *inside* the Action circuit to prove membership (check A3 of §2.13), and that circuit is arithmetic over $\mathbb{F}_{p_{\mathsf{Pallas}}}$: every wire carries one such field element. A curve-point node would carry two coordinates and an on-curve constraint at each of the 32 layers; reducing each node to its x-coordinate halves the path data the circuit threads and removes that bookkeeping. This is precisely why MerkleCRH ends in Extract: Sinsemilla yields a point, and Extract returns it to the field, so that each layer's output is directly the next layer's input and the one hash composes all the way to the root. Dropping the y-coordinate is harmless here (§2.2): the tree uses each node only as a binding digest, never to recover a child point, and a collision in the x-coordinate still reduces to one in Sinsemilla (Theorem 2.2).

Two design details are significant. The tag carries the layer index ℓ' so that a path at one height cannot replay at another. And empty leaf positions take an "uncommitted" sentinel value 2: the choice is not arbitrary — at $x = 2$ the curve equation gives $y^2 = 2^3 + 5 = 13$, and 13 is a non-square modulo both Pasta primes (the *Math Guide* verifies this), so 2 is not the x-coordinate of any Pallas point, hence can never equal a valid cmx and can never be silently confused with a genuine commitment.

Derivation of the authentication path. Alice's commitment occupies a fixed *position* pos — its index in creation order — which her wallet records when it detects the note (§2.8). Fix an anchor of any block at or after the one that mined her note, and let n be the number of leaves in that block's tree. The authentication path of leaf pos against that anchor consists of one node per level: writing $\mathsf{pos} = b_{31} \cdots b_1 b_0$ in binary, the level-i ancestor of her leaf is a left child when $b_i = 0$ and a right child when $b_i = 1$, and the path's level-i entry is that ancestor's *sibling* in the tree of size n (Figure 2). Check A3 reverses the construction: starting from cmx it combines the running node with one path entry per level — placing the node on the side b_i dictates (left when $b_i = 0$, right when $b_i = 1$) and the sibling opposite — and asserts that the final value equals the public anchor; the bits b_i and the siblings stay inside the witness.

Each entry is one of two kinds. When $b_i = 1$ the sibling subtree lies to the *left* of her leaf: it spans commitments older than hers, so every one of its positions is filled — a *complete* subtree — and, because the tree appends only on the right, its root is final and identical at every anchor. When $b_i = 0$ the sibling subtree lies to her *right*, and its value depends on the anchor: its root hashes the commitments it holds at size n, with positions not yet filled taking the uncommitted sentinel 2; a sibling subtree still entirely empty at size n contributes a per-level constant, the sentinel folded up through the layer-tagged hash. The path is therefore a deterministic function of pos and the first n public leaves, and of nothing else; the only secret is which leaf is hers.

In practice the wallet neither rebuilds the path from the leaves on each spend nor retains the whole tree. As it scans, it appends every published cmx — its own and everyone else's — but keeps only what a future path needs: the nodes from each of its own notes up to the root, the complete-subtree roots flanking those paths, and one *checkpoint* per block; the rest it prunes (this is the design of librustzcash's shardtree). It marks its note's position the moment trial decryption succeeds (§2.8). A checkpoint stores merely the position of that block's last leaf, equivalently the leaf count n — the size that fixes the block's anchor; the anchor itself is recomputed from the retained nodes on demand, never stored.

To produce the path for a spend against a chosen block, the wallet descends its retained tree to the marked leaf and, on the way back to the root, reads off each sibling subtree's root: a complete subtree contributes its stored root, a subtree lying wholly beyond the leaf count n contributes the per-level empty root, and the single subtree straddling position n is rehashed from its retained nodes. The leaf count enters only as this cutoff — every position $\geq n$ counts as empty — so one retained tree reproduces the path against any block it has checkpointed.

Nothing above requires Alice to have been online: the inputs are public, so a freshly restored wallet rebuilds the same state by replaying the chain's note commitments. Nor need it touch all 2^{32} positions, and here the tree's structure earns its keep. Cut the tree at half its depth. Below the cut lie the *shards* — the height-16 subtrees, 2^{16} leaves apiece, up to 2^{16} of them — and above it a height-16 *cap* whose own leaves are the shard roots (librustzcash sets the shard height to half the tree depth, $32/2 = 16$). A leaf's 32 siblings divide along the same cut: the low 16 (levels 0 through 15) lie inside its own shard, and the high 16 (levels 16 through 31) are cap nodes, each the root of a subtree spanning whole shards. The two halves reach Alice by entirely different routes, and that asymmetry is the economy of the scheme.

Her own shard she must hash from leaves. Its 16 low siblings are the within-shard subtrees of the descent above — complete to her left, straddling or empty to her right — so she scans that shard's commitments block by block, her own and everyone else's, and folds them; this is the one subtree whose 2^{16} leaves she handles individually. Every *other* shard she takes as a single 32-byte root. The cap's leaves are exactly those roots, which a light server streams one per completed shard (GetSubtreeRoots); Alice folds them upward with the same layer-tagged MerkleCRH, one application per cap level — the upper half of the tree's 32 layers — and reads her high siblings off the result (Figure 2 is the toy-depth analogue: the faded subtree enters only through its root F). A shard that does not yet exist — the empty right of the cap — contributes the per-level empty constant rather than a download, and her own shard she holds from leaves whether or not it has completed, so she never needs its root from the server. Her level-16 sibling is thus a neighbouring

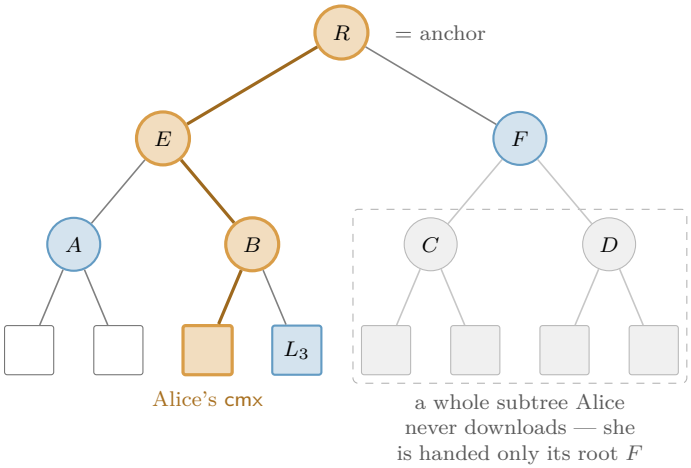

Figure 2: Computation of a Merkle authentication path (drawn at depth 3; Orchard's tree has depth 32). The leaves are the public note commitments cmx, in order. To prove her leaf (orange) is in the tree, Alice recomputes the bold route up to the root R — the *anchor* — folding in, at each level, the sibling hash beside it: the *authentication path* L_3, A, F (blue). Efficiency is the only reason not to hash the whole tree: a far subtree enters her path solely through its single root (here F), so she never downloads its leaves (faded) — the GetSubtreeRoots shortcut that lets even a freshly restored wallet witness a years-old note.

shard's root, and each sibling above it a fold of $2, 4, 8, \ldots$ roots she already holds.

A concrete case fixes the shape. Let Alice's note be the *last* leaf of the newest shard k. Below the cut, all 16 low siblings are complete left subtrees tiling the shard's earlier $2^{16} - 1$ positions, so she ingests essentially the whole shard. Above the cut, the cap's right side is empty — no shard succeeds k, so those siblings are the empty constant — while its left side folds the roots of shards $0, \ldots, k-1$, each supplied by GetSubtreeRoots. Stitching the halves — her leaf folds up to the shard-k root, which folds up the cap to the anchor — gives the full 32-level path.

The 2^{16} throughout bounds the cap's *capacity* — 2^{32} leaves over 2^{16} per shard — not Alice's traffic. She fetches one root per shard that actually exists, $\lceil n/2^{16} \rceil$ of them: a few hundred at present tree sizes, and growing by one every 2^{16} outputs. The whole history above her shard thus arrives as a few kilobytes of roots in place of millions of leaves. She performs the fold *locally*, never asking a server for one particular note's path, which would reveal which note is hers; the finished path is the private witness of check A3, and only the anchor is public.

Anchor choice as the tree grows. By the time Alice's transaction reaches a block, the live tree has advanced past her chosen anchor. This costs nothing: appending leaves creates new roots and alters no old one, so the anchor still names one fixed historical tree, her path still folds to exactly that root, and check A3 asserts exactly that equality — collision resistance of the layer-tagged hash (§2.3) makes a path to that root for a non-member infeasible. Consensus accepts the anchor of *any* main-chain block from Orchard activation onward (NU5, mainnet block 1,687,104); there is no sliding window, so proving is fully decoupled from broadcasting — Alice need not race the chain tip. The one recency caveat runs in the opposite direction: a reorganisation can strike the anchoring block from the main chain, invalidating the anchor and forcing her to rebuild the transaction; anchoring at least 100 blocks deep (Zcash's reorg limit) avoids this entirely. Deployed wallets nonetheless anchor close to the tip, by deliberate choice rather than necessity. The anchor's tree is the anonymity set in which check A3 conceals the spent note — the proof reveals only that the note is *some* leaf committed by that anchor — so the most recent admissible anchor, naming the largest such tree, maximises that set, while an idiosyncratically old or deep anchor would instead fingerprint the spender. librustzcash accordingly selects the most recent

block it has checkpointed that lies at least a fixed number of confirmations behind the tip (by default, per ZIP-315, 3 confirmations for notes the wallet produced itself and 10 for notes received from third parties), trading a sliver of reorg robustness for set size and uniformity. Consensus mandates none of this; it is wallet policy. Nor is an old anchor a soundness hole: membership in an older, smaller tree implies the note was committed even earlier, which is precisely what Alice must establish — the nullifier (§2.7), not the anchor, prevents a second spend.

2.7 Prevention of double-spending: nullifiers

The third requirement: to prevent Alice from spending the same note twice, without a public registry of spent notes (which would leak exactly what is being hidden). Each note yields a single deterministic tag — its *nullifier* — that appears only when its owner spends the note. The network maintains a set of all revealed nullifiers and rejects any repeat. The nullifier must satisfy two properties that pull in opposite directions: it must be *deterministic* (so the same note always yields the same tag, making a second spend detectable) yet *unlinkable* (so that publishing it reveals nothing about which note or address it came from).

Definition 2.7 (Orchard nullifier)**.** For a note $\mathbf{n}$ with commitment cm and the owner's nullifier key nk,

$$\mathsf{nf} := \mathsf{Extract}\big(\big[\,F_{\mathsf{nk}}(\rho) + \psi\,\big]G^{\mathsf{nf}} + \mathsf{cm}\big) \;\in\; \mathbb{F}_{p_{\mathsf{Pallas}}},$$

where $F_{\mathsf{nk}} : \mathbb{F}_{p_{\mathsf{Pallas}}} \to \mathbb{F}_{p_{\mathsf{Pallas}}}$ is a circuit-efficient PRF (Poseidon over the Pallas base field, the *Crypto Guide*), $G^{\mathsf{nf}} := \mathsf{GroupHash}^{\mathsf{Pallas}}(\texttt{z.cash} : \texttt{Orchard}, \texttt{K}) \in \mathcal{E}(\mathbb{F}_{p_{\mathsf{Pallas}}})$ is a fixed nothing-up-my-sleeve Pallas generator (§2.2), the scalar $F_{\mathsf{nk}}(\rho) + \psi$ lives in $\mathbb{F}_{p_{\mathsf{Pallas}}}$ (a valid Pallas scalar because $p_{\mathsf{Pallas}} < p_{\mathsf{Vesta}}$), and $\mathsf{Extract}$ takes the x-coordinate. (We write G^{nf} for this base to distinguish it from the spend-auth generator $G^{\mathsf{SpendAuth}}$ of §2.4.)

We read the construction against the two requirements. Determinism: every ingredient $(\mathsf{nk}, \rho, \psi, \mathsf{cm})$ is fixed once the note exists, so nf is a fixed function of the note — spending it twice yields the same nf twice, and the network rejects the second. Unlinkability: the term $[F_{\mathsf{nk}}(\rho) + \psi]\,G^{\mathsf{nf}}$ masks the commitment with a value that appears random to anyone without nk, so no one can tie the published nf back to cm or to the address (this is where the note's ρ and ψ perform their function). The security properties:

- *No double-spend* (the Orchard Book calls this *balance*, since double-spending is how one would create value): each note has exactly one nullifier, and forging a colliding nullifier for a different note reduces to DL on Pallas.
- *Spend unlinkability*: without nk, the published nf is unlinkable to the note or address, under the Decisional Diffie–Hellman (DDH) assumption on Pallas — that $([a]G, [b]G, [ab]G)$ is computationally indistinguishable from $([a]G, [b]G, [c]G)$ for independent uniform a, b, c — together with the PRF assumption on F.
- *Forward consistency (Faerie resistance)*: a freshly created note takes as its uniqueness seed the nullifier of the note spent alongside it, $\rho_{\text{new}} := \mathsf{nf}_{\text{old}}$; since the network already guarantees nullifiers are unique, this forces every note's ρ to be globally unique, closing an attack from earlier designs in which forged notes could be made to share a nullifier.

2.8 Transmission and detection of a note: encryption, trial decryption, and synchronisation

The on-chain cmx conceals the note entirely, which leaves a question the commitment alone does not resolve: how does the recipient learn of the note with which they were paid? The sender must deliver the note to them, privately, over the same public chain. Orchard achieves this with *in-band secret distribution*: every Action carries, beside its commitment, a ciphertext only the recipient can open.

Encryption (sender side). Alice holds Bob's address $(d, \mathsf{pk_d})$, with $\mathsf{pk_d} = [\mathsf{ivk}]\,\mathsf{g_d}$ and $\mathsf{g_d} = \mathsf{GroupHash}^{\mathsf{Pallas}}(\texttt{z.cash} : \texttt{Orchard-gd}, d)$ (§2.4). She forms the *note plaintext* $\mathsf{np} = (\texttt{0x02}, d, v, \mathsf{rseed}, \mathsf{memo})$, where the 32-byte rseed is the master randomness from which she derives the note's rcm, ψ, and the ephemeral secret esk below, and memo is a fixed 512-byte field of sender-chosen data. Each derivation is one $\mathsf{PRFexpand}$ call keyed by rseed, distinguished by a leading domain-separator byte and taking $\rho := \mathsf{nf}_{\text{old}}$ (as 32 little-endian bytes) as input — which binds the note's secret randomness to its Action:

$$\begin{aligned}
\psi &:= \mathrm{ToBase}\big(\mathsf{PRFexpand}_{\mathsf{rseed}}(\texttt{0x09} \,\|\, \rho)\big) \in \mathbb{F}_{p_{\mathsf{Pallas}}},\\
\mathsf{rcm} &:= \mathrm{ToScalar}\big(\mathsf{PRFexpand}_{\mathsf{rseed}}(\texttt{0x05} \,\|\, \rho)\big) \in \mathbb{F}_{p_{\mathsf{Vesta}}},\\
\mathsf{esk} &:= \mathrm{ToScalar}\big(\mathsf{PRFexpand}_{\mathsf{rseed}}(\texttt{0x04} \,\|\, \rho)\big) \in \mathbb{F}_{p_{\mathsf{Vesta}}},
\end{aligned}$$

where esk is redrawn under a fresh rseed in the rare event it vanishes. She then runs a one-shot Diffie–Hellman to a *fresh* key:

$$\mathsf{esk}\ \text{(above)}, \quad \mathsf{epk} := [\mathsf{esk}]\,\mathsf{g_d}, \quad \mathsf{s} := [\mathsf{esk}]\,\mathsf{pk_d}, \quad K_{\text{enc}} := \mathsf{KDF}(\mathsf{s}, \mathsf{epk}),$$

and seals the plaintext under an AEAD (ChaCha20-Poly1305), $C_{\text{enc}} := \mathsf{Enc}_{K_{\text{enc}}}(\mathsf{np})$. The KDF is BLAKE2b-256 with personalisation $\texttt{Zcash\_OrchardKDF}$ over $\mathsf{s}^\star \,\|\, \mathsf{epk}^\star$. Because $\mathsf{pk_d} = [\mathsf{ivk}]\mathsf{g_d}$, Bob can reach the same secret from his side as $[\mathsf{ivk}]\,\mathsf{epk} = [\mathsf{ivk}][\mathsf{esk}]\mathsf{g_d} = [\mathsf{esk}]\mathsf{pk_d}$, so, beside the sender (who knows esk), exactly the holder of ivk can recover K_{enc} from the published $(\mathsf{epk}, C_{\text{enc}})$. A second ciphertext C_{out}, sealed under the *outgoing cipher key* $\mathsf{ock} := \text{BLAKE2b-256}(\texttt{Zcash\_Orchardock},\ \mathsf{ovk} \,\|\, \mathsf{cv}^{\mathsf{net}} \,\|\, \mathsf{cmx} \,\|\, \mathsf{epk})$ (points in $\star$-encoding; $\mathsf{cv}^{\mathsf{net}}$ is the Action's net value commitment, §2.9), carries $(\mathsf{pk_d}, \mathsf{esk})$ so that the *sender* — or an auditor holding ovk — can re-derive s and read the note as well; because ock binds the Action's own $\mathsf{cv}^{\mathsf{net}}$, cmx, and epk, a C_{out} cannot be transplanted onto another Action. The Action publishes $(\mathsf{epk}, C_{\text{enc}}, C_{\text{out}})$ alongside $\mathsf{cv}^{\mathsf{net}}, \mathsf{nf}, \mathsf{rk}, \mathsf{cmx}$, with rk the randomised validating key (§2.10).

Trial decryption (recipient side). Nothing on chain marks which Action pays Bob, so his wallet *trial-decrypts every* Orchard output: for each it recomputes $\mathsf{s} := [\mathsf{ivk}]\,\mathsf{epk}$, $K_{\text{enc}} := \mathsf{KDF}(\mathsf{s}, \mathsf{epk})$, and $\mathsf{np} := \mathsf{Dec}_{K_{\text{enc}}}(C_{\text{enc}})$. The AEAD tag makes an incorrect guess fail cleanly (it returns $\perp$), so a successful open *is* the detection that the note was his. He then re-derives the full note from (d, v, rseed) — taking $\rho := \mathsf{nf}_{\text{old}}$ from the same Action — and performs the two checks that close the obvious forgeries: (i) that the sender formed the ephemeral key honestly, $[\mathsf{esk}]\mathsf{g_d} = \mathsf{epk}$ for the esk re-derived from rseed (the ZIP-212 check, defeating a mauled epk); and (ii) that the note is indeed the one the Action committed on chain, $\mathsf{Extract}\big(\mathsf{NoteCommit}_{\mathsf{rcm}}(\ldots)\big) = \mathsf{cmx}$. Only when both hold does he accept. A sender can therefore never make Bob accept a note inconsistent with the commitment.

Wallet synchronisation, including from cold storage. This is the operation a wallet performs when it restores from a seed. From the account's "birthday" height it walks the chain forward and trial-decrypts every Action; a light wallet does so against *compact* blocks, which carry only the first 52 bytes of C_{enc} — those covering the compact note plaintext: the lead byte $\texttt{0x02}$ (1 byte), d (11 bytes), v (8 bytes), and rseed (32 bytes) — together with epk, cmx, and the Action's nullifier nf, which the light wallet needs twice: it supplies $\rho := \mathsf{nf}_{\text{old}}$ for the cmx recomputation that detects the note, and it is the source of the chain's nullifier set against which spentness is checked below. The wallet records each note that opens *with its leaf position* — the index at which it appended its cmx, read directly off the block — and that position is precisely what the commitment tree of §2.6 consumes: the wallet *marks* the position in its shard tree and can then build the note's authentication path on demand, by the root-only assembly described there, without replaying every commitment. To determine which recovered notes remain spendable it additionally requires nk: it computes each note's nullifier (§2.7) and discards those whose

nullifier already appears in the chain's nullifier set. The end state is a set of unspent notes, each with a value, a position, an authentication path, and a nullifier — exactly what the Action prover of §2.11 consumes.

2.9 Conservation of value: value commitments and the binding signature

The fourth requirement: to guarantee that a transaction creates no value, while concealing every amount. The instrument is the additive homomorphism of Pedersen commitments (the *Crypto Guide*): commit to each amount, and verify that the commitments *sum* to the publicly declared net flow — the individual values remain hidden, while their balance is publicly verifiable.

Definition 2.8 (Net value commitment)**.** Each Action commits to its own value change with a Pedersen commitment

$$\mathsf{cv}^{\mathsf{net}} := [\,v_{\text{old}} - v_{\text{new}}\,]\,V^{\mathsf{Orchard}} + [\mathsf{rcv}]\,R^{\mathsf{Orchard}} \;\in\; \mathcal{E}(\mathbb{F}_{p_{\mathsf{Pallas}}}),$$

where v_{old} is the spent value, v_{new} the created value, $\mathsf{rcv} \in \mathbb{F}_{p_{\mathsf{Vesta}}}$ the commitment randomness (a Pallas scalar), and $V^{\mathsf{Orchard}} := \mathsf{GroupHash}^{\mathsf{Pallas}}(\texttt{z.cash} : \texttt{Orchard-cv}, \texttt{"v"})$, $R^{\mathsf{Orchard}} := \mathsf{GroupHash}^{\mathsf{Pallas}}(\texttt{z.cash} : \texttt{Orchard-cv}, \texttt{"r"}) \in \mathcal{E}(\mathbb{F}_{p_{\mathsf{Pallas}}})$ are nothing-up-my-sleeve points (§2.2), hence independent Pallas generators.

The spender draws each rcv uniformly at random and afresh for every Action, at the moment of bundle assembly. In contrast to the note's rcm, ψ, and esk — all derived deterministically from the note seed rseed (§2.8) — rcv belongs to no note and is never transmitted; its uniformity and secrecy are precisely what make $\mathsf{cv}^{\mathsf{net}}$ hiding, and what keep the aggregate bsk introduced below known only to the transaction author.

We now apply the homomorphism. Summing over the m Actions in a transaction,

$$\sum_{i=1}^{m} \mathsf{cv}^{\mathsf{net},i} = \Big[\textstyle\sum_i (v_{\text{old},i} - v_{\text{new},i})\Big] V^{\mathsf{Orchard}} + [\mathsf{bsk}]\,R^{\mathsf{Orchard}}, \qquad \mathsf{bsk} := \textstyle\sum_i \mathsf{rcv}_i \;\in\; \mathbb{F}_{p_{\mathsf{Vesta}}}.$$

If the transaction is balanced, the bracketed value-component equals the publicly declared net flow $v^{\mathsf{Orchard}}_{\text{balance}}$ (a signed field of the transaction; by convention a positive value is net value flowing *out* of the Orchard pool — paying fees or transparent outputs, say). The network subtracts the declared part,

$$\mathsf{bvk} := \sum_i \mathsf{cv}^{\mathsf{net},i} - [\,v^{\mathsf{Orchard}}_{\text{balance}}\,]\,V^{\mathsf{Orchard}} \;\in\; \mathcal{E}(\mathbb{F}_{p_{\mathsf{Pallas}}}),$$

and *if and only if* the transaction balances, the V^{Orchard}-component cancels and $\mathsf{bvk} = [\mathsf{bsk}]\,R^{\mathsf{Orchard}}$ is a public key whose secret bsk the transaction author knows. Proving knowledge of that secret — via a *binding signature* under key bvk — is precisely proving that the accounts balance, with no amount revealed. The binding signature is RedPallas (§2.10) instantiated on the base R^{Orchard}, the value-commitment randomness base, distinct from the spend-authorisation base $G^{\mathsf{SpendAuth}}$. A single signature over the whole transaction (concretely, over its ZIP-244 transaction digest) additionally binds all its Actions together, so that no one can remove or transplant any of them.

2.10 Authorisation of the spend: RedPallas

One capability remains: the spender must prove that they are *authorised* to spend the note — that they hold ask — and must do so without linking this spend to any other spend by the same key. RedPallas, a re-randomisable Schnorr signature, provides exactly this.

It is the Schnorr template of the *Crypto Guide* on Pallas, with hash H^* (BLAKE2b-512 reduced mod p_{Vesta}). The novel ingredient is *key re-randomisation*: given a fresh randomiser $\alpha \in \mathbb{F}_{p_{\mathsf{Vesta}}}$,

$$\mathsf{rsk} := \mathsf{ask} + \alpha \;\in\; \mathbb{F}_{p_{\mathsf{Vesta}}}, \qquad \mathsf{rk} := \mathsf{ak} + [\alpha]\,G^{\mathsf{SpendAuth}} = [\mathsf{rsk}]\,G^{\mathsf{SpendAuth}} \;\in\; \mathcal{E}(\mathbb{F}_{p_{\mathsf{Pallas}}}).$$

The spender publishes the *randomised* validating key rk, not the real ak, and signs with rsk. Because α is fresh per spend, rk is uniformly random and unlinkable across spends, yet remains a valid key for the underlying authority. The circuit enforces that rk is a re-randomisation of the witnessed ak (check A6) and that this ak is the key bound into the note's address (check A7), so that a valid Action proves the spender knows ask — without ever revealing ak.

2.11 A single shielded payment, end to end

The components now assemble into a transaction. Consider Alice paying Bob 5 ZEC ($v = 5 \cdot 10^8$ zatoshi) from a note worth 5 ZEC. The unit of shielded transfer is an *Action*, which simultaneously spends one old note and creates one new note (a deliberately fixed shape that conceals whether a given Action is "really" a spend, an output, or both). This payment is one Action.

1. Bob hands Alice an address. Bob derives a diversified address $(d, \mathsf{pk_d})$ from his ivk (§2.4) and gives it to Alice. It reveals nothing and is unlinkable to his other addresses.

2. Alice builds the Action. Alice owns an old note $\mathbf{n}_{\text{old}}$ worth 5 ZEC, whose commitment $\mathsf{cmx}_{\text{old}}$ is a leaf of the tree. She:

- computes its nullifier nf_{old} (§2.7) — she will publish this to retire the note;
- constructs Bob's new note $\mathbf{n}_{\text{new}} := (\mathsf{addr}_{\text{Bob}}, 5 \cdot 10^8, \rho_{\text{new}}, \psi_{\text{new}}, \mathsf{rcm}_{\text{new}})$ with $\rho_{\text{new}} := \mathsf{nf}_{\text{old}}$, and its commitment $\mathsf{cmx}_{\text{new}}$ (§2.5);
- forms the value commitment $\mathsf{cv}^{\mathsf{net}}$ with $v_{\text{old}} - v_{\text{new}} = 5 \cdot 10^8 - 5 \cdot 10^8 = 0$ (§2.9);
- selects a randomiser α and forms rk (§2.10);
- reads off a recent anchor and her note's Merkle path (§2.6).

3. Alice proves the Action statement. She runs the Halo 2 prover on the relation of §2.13, with public inputs $(\mathsf{anchor}, \mathsf{cv}^{\mathsf{net}}, \mathsf{nf}_{\text{old}}, \mathsf{rk}, \mathsf{cmx}_{\text{new}})$ and her note, keys, path, and randomness as private witness. The proof attests, in one shot and revealing nothing: the old note is in the tree (A3), its nullifier follows correctly (A5), she is authorised to spend it (A6, A7), the spent and created notes are well-formed (A1, A2), and the value commitment is consistent (A4).

4. Alice assembles and signs the transaction. She encrypts $\mathbf{n}_{\text{new}}$ to Bob (so that only Bob, via his ivk, can detect and open it; §2.8), attaches a SpendAuthSig under rk, and signs the whole bundle with the binding signature under bvk (§2.9), which proves balance; both signatures sign the ZIP-244 transaction sighash, welding the Action into this transaction.

5. The network verifies. Consensus checks: it recognises the anchor; nf_{old} is not already in the nullifier set (no double-spend); the Halo 2 proof verifies; the SpendAuthSig verifies under rk; the binding signature verifies under the recomputed bvk (balance). If all pass, it adds nf_{old} to the nullifier set and appends $\mathsf{cmx}_{\text{new}}$ as a new tree leaf.

6. Bob receives. Scanning with his ivk, Bob trial-decrypts the ciphertext (§2.8), recovers $\mathbf{n}_{\text{new}}$, and now owns a 5-ZEC note he can later spend by repeating the entire procedure. The chain has gained one nullifier and one commitment; an observer learns that *an* Orchard payment occurred and nothing else — not sender, recipient, or amount.

That sequence constitutes the entire protocol. The formal Action statement below is precisely the list of the in-circuit checks step 3 invokes.

2.12 What a node verifies, without ever observing the note

Note that none of the previous subsection involved the *network*. A validating node holds no viewing key, cannot trial-decrypt, and so never learns the sender, recipient, or amount of any Action. What it *can* do is confirm that the hidden data is internally consistent — and for that it relies entirely on the proof, not on the note.

What the node receives is an Orchard *bundle*, the unit into which the v5 transaction format (ZIP-225) groups all Orchard data: the per-Action public fields; one anchor shared by every Action in the bundle; a *single* aggregate Halo 2 proof covering all of the bundle's Actions; and one SpendAuthSig per Action. The declared net flow $v^{\mathsf{Orchard}}_{\mathrm{balance}}$ and the binding signature appear once per bundle, and the flags $\mathsf{enableSpends}$, $\mathsf{enableOutputs}$ are likewise bundle-level fields. Using only the public fields $(\mathsf{anchor}, \mathsf{cv}^{\mathsf{net}}, \mathsf{nf}_{\mathrm{old}}, \mathsf{rk}, \mathsf{cmx})$ of each Action, a node checks:

- the bundle's *Halo 2 proof* of the Action statement (§2.13) — this forces each spent note of nonzero witnessed value to be a genuine tree leaf (a zero-value dummy spend skips the membership check by A3's gate, by design), the nullifiers and new commitments to follow correctly, and the keys to align;
- the *anchor* is a Merkle root the node has seen on its accepted main chain (§2.6);
- $\mathsf{nf}_{\mathrm{old}}$ is *not already* in the nullifier set — the one check that cannot sit inside a single proof, because it concerns the global history (this prevents a double-spend);
- the *spend-authorisation signature* verifies under rk and the *binding signature* under the recomputed bvk (§§2.9–2.10), settling authority and balance.

If every Action passes, the node performs the two state changes of step 5 — recording each $\mathsf{nf}_{\mathrm{old}}$, appending each $\mathsf{cmx}_{\mathrm{new}}$ — which permit the next spender to prove membership and make a second spend of the same note fail. Thus the SNARK together with the two signatures enforces a payment's validity, never the inspection of the note: the node confirms that the accounts balance and that no one forged anything *while learning nothing about the note itself.* This is the precise sense in which the knowledge soundness of the Action SNARK (§3) performs all the work — the network trusts the proof exactly because a real witness extractably backs a valid one.

2.13 The Action statement, formally

Definition 2.9 (Orchard Action statement, NU5/NU6)**.** The Action SNARK proves, with public inputs

$$(\mathsf{anchor},\ \mathsf{cv}^{\mathsf{net}},\ \mathsf{nf}_{\mathrm{old}},\ \mathsf{rk},\ \mathsf{cmx},\ \mathsf{enableSpends},\ \mathsf{enableOutputs})$$

and the note, keys, Merkle path, randomiser α, and the value-commitment trapdoor rcv as private witness, the conjunction:

[A1] Old-note commitment integrity — the spent note is well-formed:

$$\mathsf{cm}_{\mathrm{old}} = \mathsf{NoteCommit}_{\mathsf{rcm}_{\mathrm{old}}}\big(\mathsf{g}^{\star}_{\mathsf{d}_{\mathrm{old}}} \| \mathsf{pk}^{\star}_{\mathsf{d}_{\mathrm{old}}} \| \mathrm{LE}_{64}(v_{\mathrm{old}}) \| \mathrm{LE}_{255}(\rho_{\mathrm{old}}) \| \mathrm{LE}_{255}(\psi_{\mathrm{old}})\big)$$

— or $\mathsf{NoteCommit}$ returns $\perp$, the specification's escape hatch for Sinsemilla's incomplete-addition exceptions (the remark following this definition).
[A2] New-note commitment integrity, with $\rho_{\mathrm{new}} := \mathsf{nf}_{\mathrm{old}}$: $\mathsf{Extract}\big(\mathsf{NoteCommit}_{\mathsf{rcm}_{\mathrm{new}}}(\ldots)\big) \in \{\mathsf{cmx}, \perp\}$ (with $\mathsf{Extract}(\perp) := \perp$) — the created note is well-formed and its uniqueness seed chains to the spent note's nullifier (Faerie resistance).
[A3] Membership, gated by v_{old} — the spent note is a real leaf: recomputing the Merkle root from $\mathsf{Extract}(\mathsf{cm}_{\mathrm{old}})$ up the witness path of length 32 gives root, and $v_{\mathrm{old}} \cdot (\mathsf{root} - \mathsf{anchor}) = 0$. (The gate by v_{old} permits a zero-value "dummy" spend to skip the membership check, which is how an Action pads with no real input.)

[A4] Net-value commitment integrity — the amounts are consistent and in range: with witnessed (magnitude, sign), $v_{\text{old}} - v_{\text{new}} = \text{magnitude} \cdot \text{sign}$, $\mathsf{cv}^{\mathsf{net}} = [v_{\text{old}} - v_{\text{new}}]V^{\mathsf{Orchard}} + [\mathsf{rcv}]R^{\mathsf{Orchard}}$, $v_{\text{old}}, v_{\text{new}} \in [0, 2^{64})$, sign $\in \{-1, +1\}$.
[A5] Nullifier integrity — the published nullifier is indeed this note's:

$$\mathsf{nf}_{\text{old}} = \mathsf{Extract}\big([F_{\mathsf{nk}}(\rho_{\text{old}}) + \psi_{\text{old}}]\, G^{\mathsf{nf}} + \mathsf{cm}_{\text{old}}\big).$$

[A6] Spend authority — the published key is a re-randomisation of the note owner's: $\mathsf{rk} = \mathsf{ak} + [\alpha]G^{\mathsf{SpendAuth}}$.
[A7] Address integrity — the keys, address, and nullifier key all belong to one identity: $\mathsf{ivk} = \bot$, or $\mathsf{ivk} = \mathsf{Commit}^{\mathsf{ivk}}_{\mathsf{r_{ivk}}}(\mathsf{Extract}(\mathsf{ak}), \mathsf{nk})$ and $\mathsf{pk_{d_{old}}} = [\mathsf{ivk}]\mathsf{g_{d_{old}}}$.
[A8] / [A9] Spend / output gating: $v_{\text{old}} \cdot (1 - \mathsf{enableSpends}) = 0$ and $v_{\text{new}} \cdot (1 - \mathsf{enableOutputs}) = 0$. The transaction builder sets these bundle-level flags; consensus rules constrain them (coinbase transactions, for instance, must set $\mathsf{enableSpends} = 0$).

A1, A2, and A7 take the specification's $\bot$-weakened forms because the circuit cannot exclude Sinsemilla's incomplete-addition exceptional cases; the relation proved is the disjunction, not the plain conjunction. The weakening costs only a negligible soundness term: producing a $\bot$ case is itself a non-trivial DL relation among the GroupHash generators (the closing sentence of the proof of Proposition 3.1), so no efficient prover exhibits one.

Each of A1–A7 is one entry from the four-requirement map of §2.1: A1–A2 represent notes, A3 proves membership, A5 (with the chain's nullifier set) prevents double-spending, A4 conserves value, and A6–A7 authorise the spender; A8–A9 stand outside the map as consensus-level gating constraints. This is the relation $\mathcal{R}_{\mathsf{Orchard}}$. Every Action is an instance of it; Halo 2 produces one aggregate proof *per bundle*, covering all of the bundle's Actions, while each Action carries its own SpendAuthSig. §2.14 sketches how the Action circuit arithmetises this relation, and §3 shows how soundness of the relation yields the protocol's security properties.

2.14 The Action circuit: arithmetisation of the relation

The Action statement of §2.13 is a *relation*; what Alice actually executes is a *circuit* that proves membership in it in zero knowledge. Halo 2 (the *Halo 2 Guide*) proves satisfiability of a fixed PLONKish circuit, so the task of the application layer is to encode the conjunction A1–A9 as one such circuit — the *Action circuit*, identical for every Action — supplied the note, keys, Merkle path, randomiser α, and the value-commitment trapdoor rcv as private witness. We sketch this encoding here; the gate-level details appear in the *halo2 Book* and the *Orchard Book*.

We assemble the circuit from a small set of reusable *gadgets* ("chips"), each a pre-wired block of custom gates and lookups realising one operation. The Orchard designers *chose* the primitives so that these gadgets are inexpensive under PLONKish arithmetisation; this motivates the use of Sinsemilla and Poseidon rather than a bit-oriented hash.

The ECC gadget. Implements the Pallas group law with the *complete* addition formulas (no edge-case failures; the *Math Guide*), together with variable- and fixed-base scalar multiplication (the former a double-and-add ladder of incomplete-addition rounds finished by a complete tail, the latter using precomputed fixed-base tables; the j-invariant-0 endomorphism serves only the recursion layer's endoscaling, not this circuit) and on-curve witnessing. It realises every group operation in the statement: $\mathsf{ak} = [\mathsf{ask}]\, G^{\mathsf{SpendAuth}}$, the re-randomisation $\mathsf{rk} = \mathsf{ak} + [\alpha]\, G^{\mathsf{SpendAuth}}$ (A6), the transmission key $\mathsf{pk_d} = [\mathsf{ivk}]\, \mathsf{g_d}$ (A7), the net value commitment $\mathsf{cv}^{\mathsf{net}} = [\, v_{\text{old}} - v_{\text{new}}\,]\, V^{\mathsf{Orchard}} + [\mathsf{rcv}]\, R^{\mathsf{Orchard}}$ (A4), and the point $[\, F_{\mathsf{nk}}(\rho) + \psi\,]\, G^{\mathsf{nf}} + \mathsf{cm}$ inside the nullifier (A5).

The Sinsemilla gadget. Implements Sinsemilla (§2.3) by realising its generator table $P[m]$ as a Halo 2 *lookup* (the lookup argument of the *Halo 2 Guide*): each 10-bit message chunk costs one table lookup and two incomplete additions. On it rest the two composed commitment gadgets

the Orchard Book names explicitly — NoteCommit, the in-circuit NoteCommit proving A1 and A2, and CommitIvk, the in-circuit short commitment proving $\mathsf{ivk} = \mathsf{Commit}^{\mathsf{ivk}}_{\mathsf{r_{ivk}}}(\mathsf{Extract}(\mathsf{ak}), \mathsf{nk})$ in A7 — each bundling the Sinsemilla rounds with the bit-decomposition and canonicity checks of their field inputs.

The Merkle-path gadget. Stacks 32 layer-tagged MerkleCRH hashes (themselves Sinsemilla short hashes, §2.6); at each layer a conditional swap orders the running node and its witnessed sibling by one bit of the leaf position, and the gadget asserts the recomputed root equal to the public anchor — check A3, gated by v_{old} so that a dummy spend may skip it.

The Poseidon gadget. Implements the Poseidon permutation (*Crypto Guide*), whose only non-linearity is $x \mapsto x^5$, to evaluate the nullifier PRF $F_{\mathsf{nk}}(\rho) = \mathsf{PoseidonHash}(\mathsf{nk}, \rho)$ cheaply in-circuit; its output feeds the ECC gadget for A5.

Decomposition, range, and boolean gadgets. Lookup-based gadgets split field elements into the bit-chunks the other gadgets consume and range-check values: that $v_{\text{old}}, v_{\text{new}} \in [0, 2^{64})$ and sign $\in \{-1, +1\}$ (A4), and that point and field encodings are canonical. (The flags enableSpends, enableOutputs are verifier-supplied public inputs — ZIP-225's flagsOrchard byte fixes them to bits outside the circuit — and enter the circuit only through the plain-gate products of A8–A9.)

Structure of the proof. These gadgets compose into a single fixed circuit whose native field is the Pallas base field $\mathbb{F}_{p_{\mathsf{Pallas}}}$ — the field in which Pallas coordinates reside, so that the ECC gadget runs natively — proved with the Halo 2 inner-product argument over the Pasta cycle. The circuit uses $2^{11} = 2048$ rows (the *Halo 2 Guide*'s circuit-size exponent $k = 11$ — not to be confused with Sinsemilla's chunk width $k = 10$, Construction 2.1), with ten advice columns beside the fixed columns and the Sinsemilla lookup table (the *Halo 2 Guide*). Every Action, whatever it spends or creates, is one instance of this same circuit; a bundle's Actions are proved together in one aggregate proof that a node checks (§2.12) from the public inputs alone, and the accumulation machinery of the *Halo 2 Guide* lets a node verify many such proofs at amortised cost approaching that of a single verification.

3 End-to-end security: derivation of Orchard's properties from the SNARK

The Halo 2 SNARK on the Action relation, together with the algebraic structure of the Orchard primitives, produces the security properties that render shielded payments functional. We establish those connections below.

3.1 Guarantees provided by a valid Action proof

A SNARK π accepted on public inputs $(\mathsf{anchor}, \mathsf{cv}^{\mathsf{net}}, \mathsf{nf}_{\text{old}}, \mathsf{rk}, \mathsf{cmx}, \ldots)$ testifies, with overwhelming probability, that the prover knows a witness satisfying A1–A9. By knowledge soundness of Halo 2 (the *Halo 2 Guide*), an extractor can recover this witness from any successful prover. The soundness-side properties below (nullifier uniqueness, spend authority, balance) each follow from one or more conjuncts of A1–A9 plus the algebraic properties of the primitives; §3.2 first supplies the primitive reduction those arguments share, and privacy rests instead on the zero knowledge of the SNARK (§3.6) together with the DDH, Poseidon-PRF and KDF/AEAD assumptions that Theorem 3.5(iv) adds.

3.2 Reduction of Sinsemilla collision-resistance to DL on Pallas

Proposition 3.1 (Sinsemilla CR $\Leftarrow$ DL)**.** *A collision $M \neq M'$ between messages of the same chunk count ℓ with $\mathsf{Sinsemilla}_D(M) = \mathsf{Sinsemilla}_D(M')$ yields a non-trivial DL relation among the generators $\{P[0], \ldots, P[2^k - 1]\}$.*

Sketch. Unfolding the Sinsemilla update (in the case no incomplete-addition exception occurs), with ℓ the number of k-bit chunks:

$$\mathsf{Sinsemilla}_D(M) = 2^\ell \cdot Q + \sum_{i=1}^{\ell} 2^{\ell-i} \cdot P[m_i].$$

A collision gives $\sum_i 2^{\ell-i}(P[m_i] - P[m'_i]) = 0$, i.e. a non-trivial linear relation over $\mathbb{F}_{p_{\mathsf{Vesta}}}$ on the generators. Since hash-to-curve modelled as a random oracle samples these independently, finding such a relation is the multi-generator discrete-log relation problem, and no efficient adversary produces one without solving DL (the *Crypto Guide* gives this reduction in its Pedersen-hash collision analysis). For differing chunk counts $\ell \neq \ell'$, the unfolded equality instead gives $(2^\ell - 2^{\ell'})\,Q + \sum_i \cdots = \mathcal{O}$ with $2^\ell \not\equiv 2^{\ell'} \pmod{p_{\mathsf{Vesta}}}$ for all admissible lengths, i.e. a non-trivial DL relation among $\{Q\} \cup \{P[0], \ldots, P[2^k - 1]\}$; since Q is itself a $\mathsf{GroupHash}$ output modelled as an independent random generator, this case reduces to DL identically. The incomplete-addition exceptions are themselves non-trivial DL relations among the same generators and so also negligible. □

Sinsemilla CR is the primitive ingredient for note-commitment binding: two distinct notes cannot share a commitment, so A1 and A2 are genuine integrity statements about $\mathsf{cm}_{\mathrm{old}}$ and $\mathsf{cm}_{\mathrm{new}}$. Combined with knowledge soundness, this entails that the prover knows the specific opening of each commitment.

3.3 Nullifier uniqueness

Proposition 3.2 (Nullifier uniqueness)**.** *Two distinct notes $\mathbf{n} \neq \mathbf{n}'$ cannot share a nullifier except with probability negligible in the DL hardness of Pallas (treating Poseidon outputs as random scalars under the PRF assumption).*

Sketch. A collision $\mathsf{nf}(\mathbf{n}) = \mathsf{nf}(\mathbf{n}')$ in x-coordinate gives $[F_{\mathsf{nk}}(\rho) + \psi]G^{\mathsf{nf}} + \mathsf{cm} = \pm([F_{\mathsf{nk}}(\rho') + \psi']G^{\mathsf{nf}} + \mathsf{cm}')$, a non-trivial linear relation among $G^{\mathsf{nf}}, \mathsf{cm}, \mathsf{cm}'$. Expanding $\mathsf{cm}, \mathsf{cm}'$ via their known openings (extracted by knowledge soundness, or supplied by the adversary who forged the notes) over the Sinsemilla generators and the blinding base H_D turns this into a known non-trivial DL relation among the independent $\mathsf{GroupHash}$ outputs $\{G^{\mathsf{nf}}, Q, P[0], \ldots, P[2^k - 1], H_D\}$ — distinctness of the notes forces some coefficient mismatch — and producing such a relation reduces to DL on Pallas as in Proposition 3.1. □

A5 (nullifier integrity) plus knowledge soundness ensures that $\mathsf{nf}_{\mathrm{old}}$ in the public inputs is genuinely the nullifier of some specific committed note, not an arbitrary value. The chain's duplicate-nullifier rejection then prevents double-spending.

3.4 RedPallas unforgeability and re-randomisation

Proposition 3.3 (RedPallas EUF-CMA $\Leftarrow$ DL)**.** *RedPallas is EUF-CMA-secure assuming DL on Pallas in the random-oracle model. The re-randomisation extension preserves security: a forger that, given ak and signatures under re-randomised keys, produces a valid signature under $\mathsf{rk} = \mathsf{ak} + [\alpha]\,G^{\mathsf{SpendAuth}}$ for an α of its own (possibly ak-dependent) choice yields a DL solver for ak.*

Sketch. The standard forking argument (the *Crypto Guide*, via the Bellare–Neven general forking lemma) reduces a Schnorr forger to a DL adversary, and it covers re-randomisation because RedPallas is key-prefixed: the challenge $H^*(R \,\|\, \mathsf{rk} \,\|\, m)$ hashes the verification key, so response-shifting cannot transport signatures between keys. The reduction embeds the DL challenge as $\mathsf{ak} := X$. It answers signing queries under any $\mathsf{rk}_i = \mathsf{ak} + [\alpha_i]\, G^{\mathsf{SpendAuth}}$ without knowing any secret, via the honest-verifier zero-knowledge simulator with random-oracle programming: sample c, s uniformly, set $R := [s]\, G^{\mathsf{SpendAuth}} - [c]\, \mathsf{rk}_i$, and program $H^*(R \,\|\, \mathsf{rk}_i \,\|\, m) := c$, aborting on programming collisions exactly as in the *Crypto Guide*'s Schnorr EUF-CMA theorem. It then forks the adversary at the forgery's critical query $H^*(R^* \,\|\, \mathsf{rk}^* \,\|\, m^*)$, obtaining two accepting transcripts that share R^* — and, since both forks share the critical query input, the same rk^*, hence the same α^* — with distinct challenges; 2-special soundness extracts rsk^*, the discrete logarithm of rk^*, and the output $\mathsf{ask} = \mathsf{rsk}^* - \alpha^*$, with α^* the adversary-supplied randomiser, solves DL for ak. Separately, and only for honest signer-chosen uniform α: sampling α at random and back-deriving $\mathsf{ak} := \mathsf{rk} - [\alpha]\, G^{\mathsf{SpendAuth}}$ reproduces the view of $(\mathsf{ak}, \alpha, \mathsf{rk})$ exactly, so honest re-randomisations are distributed as plain Schnorr keys — the unlinkability statement of the *Crypto Guide*, which does not by itself cover adversary-chosen α. □

A6 (spend authority) plus knowledge soundness ensures the prover knows α relating rk to ak; combined with the SpendAuthSig under rk at the transaction layer, this enforces that the spender knows ask corresponding to ak, hence to the address $\mathsf{pk}_{\mathsf{d}_{\mathrm{old}}}$ via A7.

3.5 Balance preservation via the Pedersen homomorphism

The combination of A4 (per-Action value commitment integrity), the Pedersen homomorphism on $\mathsf{cv}^{\mathsf{net}}$, and the binding signature on bvk gives end-to-end balance:

Proposition 3.4 (Balance)**.** *Any accepted Orchard bundle has* $\sum_i (v_{\mathrm{old},i} - v_{\mathrm{new},i}) = v^{\mathsf{Orchard}}_{\mathrm{balance}}$ *except with probability negligible in the DL hardness of the Pasta curves in the random-oracle model (Vesta for the knowledge soundness of the Action SNARK supplying the A4 openings; Pallas for the binding-signature extraction and the independence of the value-commitment bases).*

Sketch. By A4 and knowledge soundness, each $\mathsf{cv}^{\mathsf{net},i}$ commits to the true per-Action delta $v_{\mathrm{old},i} - v_{\mathrm{new},i}$. The bundle sum $\sum_i \mathsf{cv}^{\mathsf{net},i}$ equals $[\sum_i (v_{\mathrm{old},i} - v_{\mathrm{new},i})]V^{\mathsf{Orchard}} + [\sum_i \mathsf{rcv}_i]R^{\mathsf{Orchard}}$ by homomorphism. Consensus computes bvk and checks the binding signature; a valid binding signature is a Fiat–Shamir proof of knowledge of $\log_{R^{\mathsf{Orchard}}} \mathsf{bvk}$, so the forking extraction in the proof of Proposition 3.3 recovers bsk with $\mathsf{bvk} = [\mathsf{bsk}]\, R^{\mathsf{Orchard}}$; combined with the openings extracted via A4, a non-zero V^{Orchard}-component of bvk would yield a discrete-log relation between the independent bases V^{Orchard} and R^{Orchard}, contradicting DL — hence the value-component equals $[v_{\mathrm{balance}}]V^{\mathsf{Orchard}}$ and the total value flow matches the declared transparent balance. □

This is the fundamental conservation property: shielded transactions cannot mint zatoshi out of nothing.

3.6 Zero knowledge of the SNARK

Halo 2 is statistical zero knowledge in the random-oracle model (the *Halo 2 Guide*'s zero-knowledge analysis). The simulator on input the instance $(\mathsf{anchor}, \mathsf{cv}^{\mathsf{net}}, \mathsf{nf}_{\mathrm{old}}, \mathsf{rk}, \mathsf{cmx}, \ldots)$ operates as follows:

1. Sample the Fiat–Shamir challenges $\theta, \beta, \gamma, y, x, x_1, x_2, x_3, x_4$ and the IPA challenges $u_1, \ldots, u_k$ ($k = 11$ rounds, the circuit-size exponent) uniformly, programming the random oracle to return these on the relevant transcript prefixes.

2. Sample the final IPA scalar a uniformly.

3. Use the freedom in the blinding terms ℓ_j, r_j (added at each IPA round) and the polynomial blinders (added to each committed prover polynomial: the advice columns in phase 1, the running products in phase 3, and the quotient chunks in phase 4) to satisfy the verifier's equations algebraically without knowing the witness.

The resulting transcript is statistically indistinguishable from a real proof; the only gap from perfect ZK is the random-oracle programming, which a verifier that observes only the transcript cannot detect.

ZK supplies the proof's share of Orchard's privacy: the SNARK transcript reveals nothing beyond the public inputs of each Action. That the published data leak nothing further is a separate, computational matter: the nullifier — itself a public input (§2.7) — hides its note under the PRF security of Poseidon, and the epk and ciphertexts published alongside the public inputs (§2.8) hide recipient and value under the DDH and KDF/AEAD assumptions that Theorem 3.5(iv) adds.

3.7 Composition of end-to-end soundness

The full SNARK soundness composes three ingredients:

- *PIOP soundness.* Schwartz–Zippel error per polynomial-identity check, summed across constraints. For $\mathbb{F} = \mathbb{F}_{p_{\mathsf{Pallas}}}$ with $|\mathbb{F}| \approx 2^{254}$ and constraint degree $\leq d_{\max}$, the error per check is $\leq d_{\max}\, n/|\mathbb{F}| \approx d_{\max} \cdot 2^{-243}$ (the gate polynomial composes degree-$d_{\max}$ gates with column polynomials of degree $n-1$, $n = 2^{11}$; a cheating prover's recombined quotient $h \cdot Z_H$ has degree $< d_{\max} n$), overwhelmingly small.

- *PCS extractability.* Reduces to DL on the IPA curve (the *Halo 2 Guide*), with the accumulation analysis (the *Halo 2 Guide*) covering the deferred-MSM batching by which a node amortises verification across proofs (§2.14); the same analysis would carry soundness across recursion, which Orchard's deployment does not use.

- *Fiat–Shamir soundness in the ROM.* The *Crypto Guide*'s FS theorem covers Σ-protocols, losing roughly a factor Q in the prover's RO-query budget; for the $\log d$-round Halo 2 transcript the naive multi-round analysis loses $q^{O(\log d)}$ (the *Halo 2 Guide*), a bound vacuous at $Q = 2^{80}$. The negligible-error conclusion rests on the tighter treatments the *Halo 2 Guide* presents: BCMS20's direct ROM+DL analysis, or Ghoshal–Tessaro's tight Fiat–Shamir bound in the AGM.

Each step's soundness error is negligible at Orchard's parameters ($|\mathbb{F}| \approx 2^{254}$, $\lambda = 127$ under the group-order-$\approx 2^{2\lambda}$ convention; effectively ≈ 126 bits after the automorphism speedups, the *Crypto Guide*); the union bound across steps is itself negligible. Combined with the application-level reductions (Propositions 3.1–3.4), the full chain gives:

Theorem 3.5 (Orchard end-to-end security, informal)**.** *Under DL on both Pasta curves (Pallas for the application-level primitives, Vesta for the IPA commitment) in the random-oracle model — and, for property (iv), additionally DDH on Pallas, the PRF security of Poseidon (§2.7), and the KDF/AEAD securing the note ciphertext (§2.8) — no efficient adversary can (i) produce an accepted Action without knowing a witness satisfying A1–A9 — in particular, any Action whose witnessed v_{old} is nonzero must spend a genuine leaf of the commitment tree (zero-value dummy spends, A3's gate, are accepted by design), (ii) cause two Actions to share a nullifier without spending the same note twice, (iii) cause a bundle's declared transparent balance $v_{\text{balance}}^{\mathsf{Orchard}}$ to differ from its true net value flow $\sum_i (v_{\text{old},i} - v_{\text{new},i})$ — in particular, to exceed it, which would create value (Proposition 3.4), or (iv) link a published Action to any specific recipient or value beyond what is explicit in the public inputs.*

The four properties are, respectively, soundness of the Action SNARK, nullifier uniqueness, balance preservation, and zero knowledge. The first three reduce (informally) to DL on the Pasta curves in the ROM (Pallas for the application-level reductions, Vesta for the IPA extraction). For the fourth (privacy), the zero knowledge of the SNARK transcript is statistical — given the perfectly-hiding Pedersen and Sinsemilla blinders and the random-oracle programming, it does not rest on DL — but full unlinkability of an Action additionally rests on the DDH assumption on Pallas and the PRF security of Poseidon (§2.7) and on the KDF/AEAD securing the note ciphertext (§2.8), and is therefore computational. The asymmetry within the proof itself is typical of the discrete-log SNARK family: soundness is computational (a cheating prover is only *bounded*, not impossible), whereas the zero knowledge of the transcript is information-theoretic.

4 The Zcash proof-system landscape

This section closes with a survey of the proof systems Zcash has used across its three generations of shielded payments. Concrete trade-offs drove each transition: trusted setup risk, recursion compatibility, post-quantum posture, and on-chain efficiency.

4.1 Sprout (2016–present, deprecated)

The original shielded protocol used a *BCTV14* SNARK (Ben-Sasson, Chiesa, Tromer, Virza) over the pairing-friendly curve BN254, with constant-size proofs ($\sim$ 290 bytes) and constant verification time. Its parameters came from a per-circuit trusted setup: the 2016 ceremony, a six-party multi-party computation that generated the BCTV14 proving and verification keys for the fixed Sprout circuit, sound provided at least one participant destroyed their secret share. Sprout demonstrated that zk-SNARK shielded payments were feasible at scale, but it remained a transitional design: the prover was slow, and the circuit hashed with SHA-256 — a bit-oriented hash whose boolean operations are non-native to arithmetic constraints, hence costly in-circuit. Decisively, a flaw in BCTV14 discovered in 2018 would have allowed undetected counterfeiting; the pool was deprecated and shielded Sprout value was migrated to later pools.

4.2 Sapling (2018–present)

Sapling (the second network upgrade) replaced this stack with Groth16 — 192-byte proofs, three-pairing verification — over BLS12-381, with the embedded curve Jubjub serving the in-circuit group operations and a Pedersen hash serving the Merkle tree. The result was a far faster prover, a smaller proof, and cheaper in-circuit hashing. Two structural limitations remained. Groth16 still requires a per-circuit trusted setup — the 2018 Sapling MPC, whose universal first phase was the Powers of Tau ceremony with many participants — and no known elliptic-curve cycle extends BLS12-381, so the proving system admits no efficient recursion.

4.3 Orchard (2022–present, NU5)

Orchard (NU5) replaced the Sapling stack with the Halo 2 / Pasta stack developed in the *Halo 2 Guide*. The principal choices are:

- Halo 2: PLONKish PIOP, IPA-PCS, accumulation-friendly. No trusted setup.
- Pasta cycle: Pallas as the application curve, with Vesta as the proving curve in the IPA layer. Designed for recursion.
- Sinsemilla hash: an algebraic hash whose collision resistance reduces to DL on Pallas — an assumption the protocol already requires elsewhere — and whose chunk-table structure maps directly onto Halo 2's lookup argument (§2.3).

- Poseidon: serves the nullifier PRF (§2.7), where a keyed in-circuit PRF rather than a CRH is required. Unlike Sinsemilla, its security rests on cryptanalytic confidence in algebraic-attack resistance rather than on a hardness assumption such as DL.

The shift followed a consistent design principle: "transparent SNARK, recursion-ready" constituted the design target, and the designers engineered the Pasta cycle to fit.

4.4 Comparative summary

	Sprout	Sapling	Orchard
SNARK	BCTV14	Groth16	Halo 2
Curve	BN254	BLS12-381 + Jubjub	Pallas + Vesta
Trusted setup	Per-circuit (six-party MPC)	Per-circuit (large MPC)	None
Recursion-ready	No	No	Yes (via Pasta cycle)
Proof size	~290 B	192 B	~5 kB (one Action)
Verification time	Constant (pairing)	Constant (3 pairings)	$O(\log n)$ amortised

Table 1: Three generations of Zcash shielded-payment proof systems. Orchard verification: $O(\log n)$ amortised ($n = 2^{11}$ rows; per-proof succinct checks, the linear MSM batched across proofs — the *Halo 2 Guide*).

The proof size grew between Sapling and Orchard; this is the cost of transparency (a one-Action bundle's proof occupies $2720 + 2272 = 4992$ bytes in the v5 encoding, including the $2\log_2 2^{11} = 22$ group elements of the IPA rounds). The benefit, beyond the removal of trusted setup, is the foundation for recursion: future generations (Tachyon, eventual rollup designs) can build on Orchard's foundation without revisiting the cryptographic stack.